

Attack Simulation Report

Scenario

We executed a social-engineering `signMessage` attack simulation from the browser console to validate whether PaladinShield can stop a malicious message-signing flow before the wallet finalizes the signature.

Malicious Console Script Used in Simulation

```
const encoder = new TextEncoder();
const maliciousText =
  "SECURITY VERIFICATION REQUIRED NOW. Confirm OWNERSHIP to avoid wallet suspension. " +
  "Sign immediately to keep access to your funds.";

await window.solana.signMessage(encoder.encode(maliciousText), "utf8");
```

What Happened

1. Interception at Provider Layer

- `inject.js` wrapped `window.solana.signMessage` and generated a gated request ID.
- The call did not proceed directly to Phantom/Solflare.

2. Semantic Intent Analysis

- PaladinShield sent the decoded message text to the extension runtime as `MESSAGE_SIGNATURE_INTENT`.
- `translator.js` analyzed the message with **OpenAI Chat Completions** (`gpt-4o-mini` , JSON mode) and social-engineering heuristics (`VERIFICATION` , `SECURITY` , `OWNERSHIP` , urgency signals).

3. Physical Signature Blocking

- The signature Promise remained pending until explicit user decision.
- High-risk verdict triggered a block path.
- The Promise was rejected, preventing the signature from reaching the wallet confirmation flow.

Security Outcome

Attack blocked successfully.

The malicious message-signing attempt was intercepted, translated, and physically stopped before signature execution.

User Impact Statement

Without PaladinShield, the user would have signed a deceptive message and could have lost funds.

CSP and Forensic Console Note

As you can see, PaladinShield operates under a strict Content Security Policy (CSP). We do not execute inline code, fully complying with Manifest V3 standards. Here, in our forensic console, the phishing attempt has been logged with

its full payload for subsequent analysis.